

ANATOMY OF DISTRO

HOW A FROM-SCRATCH GLIBC LINUX IS PUT TOGETHER, PIECE BY PIECE

AUTHOR

Javier Fernández

CONTACT

jfernand@me.com

ISSUED

2026-09-14

ABSTRACT

distro is a Rust CLI that builds a bootable, glibc-based Linux system entirely from upstream source, using nothing but the host's own compiler toolchain. This report explains how the thing is actually put together: what runs first when the machine powers on, what each binary and library in the image is for, how they depend on one another, and how the build pipeline turns forty-odd separate upstream projects into one disk image. It is organized the way the system itself is layered — kernel, init, a static POSIX base, the seat/session daemons, and the still-unused Wayland-core libraries waiting for a compositor — rather than as a chronological log of work done.

DOCUMENT CONTROL

WORKSPACE	Cargo workspace: builder-core (lib) · distroless (musl/BusyBox) · distro (glibc/from-scratch) · distro-init (PID 1)
TARGET	Native x86_64-unknown-linux-gnu — host toolchain, no cross-compilation
COVERAGE	Everything built and QEMU-verified through Phase 2 — a real kernel, real login, seat/session daemons, Wayland-core libraries
NOT COVERED	Phases 3–6: a compositor, GPU drivers, a Rust toolchain on-target, COSMIC itself — see §7

01

WHAT DISTRO IS

Most ways to get a custom Linux system booting quickly start from someone else’s finished distribution — `debootstrap`, a container base image, an Arch `pacstrap` — and layer changes on top. This project rejected that: every binary in the built image is compiled here, from source, against nothing but the host machine’s own `glibc` and `gcc`. The one exception is the same one every real distribution makes: the host’s **compiler toolchain** — `gcc`, `binutils`, `meson`, `autoconf` — is infrastructure, not distro content, the same way a bootstrap compiler is infrastructure for a self-hosting language.

CRATE	ROLE
<code>builder-core</code>	Shared library: kernel build (<code>FEATURE_PACKS</code>), disk-image assembly, the QEMU test harness, the USB writer. Used by both distros unchanged.
<code>distroless</code>	A separate, minimal distro built by this same workspace: <code>musl</code> + <code>BusyBox</code> + <code>utils</code> , cross-compiled. Not covered here.
<code>distro</code>	The subject of this report: a from-scratch <code>glibc</code> system, native-compiled, aimed eventually at a COSMIC desktop.
<code>distro-init</code>	A from-scratch PID 1 written for this project, ~100 lines of Rust — see §2.2.

- § 2
- The boot sequence — what actually happens, in order, from power-on to a shell.
- § 3
- The kernel — the config baseline, the feature packs, and how to pick a version.
- § 4
- The static base: `coreutils`, `shell`, `login` — one binary in, no shared-library bookkeeping.
- § 5
- The seat/session layer: `seatd`, `dbus`, `eudev` — what each one actually does.
- § 6
- The Wayland-core libraries — built, installed, not yet used by anything.
- § 7
- How it’s all actually built: the pipeline, the config file, the `sysroot`.
- § 8
- What isn’t part of the picture yet.

02

THE BOOT SEQUENCE

The clearest way to see how the pieces fit is to follow what actually happens, in order, when the built image boots. At a high level, every Linux system does the same three things: a **bootloader** (`GRUB`) finds and loads the **kernel**; the kernel initializes hardware, mounts the root filesystem, and executes exactly one program as process ID 1 — conventionally called **init**; and everything else on the system is, directly or indirectly, started by that init process. What differs system to system is entirely what init actually does next — `systemd` starts hundreds of units in dependency order; `BusyBox`’s init reads a small `/etc/inittab`; this project writes its own init (§2.2) that does five things and nothing else.

STEP	COMPONENT	WHAT HAPPENS
1	<code>GRUB</code>	Reads <code>/boot/vmlinuz</code> and hands off. Installed by <code>builder-core</code> ’s image stage, unchanged from <code>distroless</code> .
2	<code>Kernel</code>	Boots, mounts the <code>ext4</code> rootfs read-write, execs <code>/sbin/init</code> . Built by <code>builder-core::stages::kernel</code> — the same generic stage <code>distroless</code> uses.
3	<code>distro-init</code>	Our own PID 1 (§2.2). Mounts <code>/proc</code> , <code>/sys</code> , <code>/dev</code> (<code>devtmpfs</code>), and a fresh <code>tmpfs</code> at <code>/run</code> .
4	<code>udev</code> , <code>seatd</code> , <code>dbus-daemon</code>	Forked and exec’d, in that order. <code>distro-init</code> then runs <code>udevadm trigger</code> once — a coldplug that tells <code>udev</code> about the devices <code>devtmpfs</code> already created before it started.
5	<code>agetty</code> ×2	Forked on <code>tty1</code> (VGA console) and <code>ttyS0</code> (serial, for QEMU/headless use). Whichever exits gets respawned.

STEP	COMPONENT	WHAT HAPPENS
6	login	<code>agetty</code> execs <code>/bin/login</code> once a username is typed. <code>login</code> reads <code>/etc/passwd</code> + <code>/etc/shadow</code> for real — the <code>root</code> account's shadow entry has an empty password field, so <code>login</code> skips the password prompt entirely rather than reimplementing BusyBox's separate no-password shortcut.
7	bash	<code>login</code> execs the shell named in <code>/etc/passwd</code> . A real, interactive shell — not a script, not a fallback.

THIS IS NOT A DIAGRAM — IT WAS WATCHED HAPPENING

Every arrow above was reproduced live in QEMU with a scripted serial-console session, not inferred from source reading: `login: root` → no password prompt → `-bash-5.2#` → `dbus-send --system ... ListNames` returns a real reply → `udevadm info --query=all --name=/dev/tty1` returns a populated device entry. See §7.4 for the harness.

2.1 WHY A CUSTOM INIT AT ALL

BusyBox (which `distroless` uses) ships its own tiny init built in; nothing glibc-based has an equivalent single binary. Rather than reach for `systemd` — which would drag in `logind`, `udev`, `journald`, and a great deal more than this system currently needs — `distro-init` is ~100 lines of Rust using the `nix` crate:

DISTRO-INIT/SRC/MAIN.RS — THE WHOLE SUPERVISION LOOP, ABBREVIATED

```
fn main() {
    mount_basic_filesystems();           // proc, sysfs, devtmpfs, tmpfs at /run

    let mut udevd_pid = spawn_udevd();   // no -d: stays a tracked child, not a daemon
    let mut seatd_pid = spawn_seatd();
    let mut dbus_pid = spawn_dbus();     // --nofork, same reasoning
    coldplug_devices();                  // `udevadm trigger`, one-shot

    let mut ttyl_pid = spawn_ttyl();     // agetty
    let mut serial_pid = spawn_serial();

    loop {
        match waitpid(None, Some(WaitPidFlag::empty())) {
            Ok(WaitStatus::Exited(pid, _)) | Ok(WaitStatus::Signaled(pid, _, _)) => {
                // whichever of the five died gets forked again; anything
                // else reaped here was just an orphaned grandchild.
            }
            ...
        }
    }
}
```

Every daemon it starts would normally daemonize itself (fork, detach, exit the parent) — `dbus-daemon --nofork` and `udev` with `no -d` suppress that, so they stay `distro-init`'s direct children and its `waitpid` loop actually sees them exit if they crash.

03 THE KERNEL

Step 2 of §2's boot table — the kernel itself — is built by `builder-core::stages::kernel`, the same code `distroless` uses unchanged. `distro`'s own `distro.toml` names a version and a download URL (currently 6.17, from kernel.org) under `[kernel]`; `distro build-kernel` fetches, configures, and compiles it.

3.1 WHERE THE STARTING CONFIG COMES FROM

There are two starting points, chosen by whether `kernel.config_file` is set in the config file:

- **No `config_file` (the default, and what `distro` currently uses).** Runs `make defconfig` — the kernel's own “reasonable defaults for this architecture” config — and then strips it down: every option belonging to one of the named feature packs below gets turned off, regardless of whether `defconfig` turned it

on. Everything `defconfig` sets that **isn't** owned by a pack (PCI, ACPI, EFI boot, block/ATA/virtio, ext4/vfat, the console) is left exactly as `defconfig` set it — only the packs are deliberately stripped, not the boot-essential baseline.

- **config_file set.** A previously saved `.config` (produced by an interactive `make menuconfig` session — see below) is used as-is, only re-resolved via `make olddefconfig` in case it predates a kernel upgrade. Nothing gets stripped: whoever hand-picked that config already decided what they wanted.

Either way, whatever packs are named in `kernel.features` get turned back on as the final step — so even a hand-edited `config_file` can still request named packs on top of it, without needing to know the underlying `CONFIG_*` symbols.

3.2 THE FEATURE PACKS

Fourteen named bundles, each just a curated list of Kconfig options one `kernel.features` entry away from being switched on as a unit:

PACK	WHAT IT TURNS ON
graphics	DRM/KMS graphics + fbdev console (i915, virtio-gpu, bochs, AGP) instead of plain VGA text.
sound	ALSA sound subsystem and the Intel HDA driver.
wireless	The Wi-Fi stack (cfg80211/mac80211) and rkill.
hid-extras	Per-vendor HID quirk drivers (Sony, Samsung, Gyration, ...) and the hiddev/hidraw userspace interfaces — generic USB HID keyboards/mice work without this.
legacy-nics	Dedicated Ethernet chipset drivers (Tigon3, Tulip, E100/E1000(E), Sky2, Forcedeth, 8139too, R8169) for real hardware — QEMU's virtio-net always works without this.
legacy-buses	PCMCIA/CardBus (Yenta) and legacy PATA chipset drivers — AHCI/virtio-blk always work without this.
network-fs	NFS (client + root-over-NFS), 9P, and autofs.
netfilter	Connection tracking, NAT, and iptables — only useful if this box routes or firewalls traffic.
security-extras	Disk quotas, POSIX ACLs, and SELinux.
iommu	AMD/Intel IOMMU support — only needed for PCI passthrough or running this as a virtualization host.
debug	Kernel debug instrumentation (Magic SysRq, schedstats, block-IO tracing, boot-param/entry debug) — useful while bringing up boot, dead weight once stable.
ia32-emulation	Run 32-bit x86 binaries on this 64-bit kernel.
iso9660	ISO9660/Joliet/zisofs filesystem support, for booting or mounting optical media images.
boot-logo	Framebuffer console + boot-time Linux logo (the stock penguin, or a custom image via <code>kernel.logo_file</code> — must be an 80×80, ASCII/P3 PPM with at most 224 colors, the kernel's own logo converter's hard requirement).

DISTRO'S CURRENT KERNEL.FEATURES: NONE

`distro.toml` sets no `kernel.features` at all — every pack above is off, which is why `distro`'s kernel today is the fully stripped `defconfig` baseline. Phase 3 (Mesa/graphics) will need at least `graphics` turned on; the original project roadmap expects most of these packs on for a real desktop eventually, none of that has happened yet.

3.3 PICKING A VERSION, AND HAND-TUNING THE CONFIG

`builder-core::stages::kernel` also provides `resolve_kernel()` (looks up the current stable or long-term-support release from kernel.org's own release feed and writes its version/URL into the config file) and `menuconfig()` (runs an interactive `make menuconfig` against the fetched sources, then saves the result for reuse as `kernel.config_file`) — both generic, shared code. `distroless`'s

CLI exposes them directly (`distroless resolve-kernel --channel stable`, `distroless menu-config --save-to kernel.config`, `distroless list-features`); `distro`’s CLI currently does not wire up equivalent subcommands, so picking a kernel version or hand-tuning the config for `distro` means editing `distro.toml` directly rather than going through a CLI command — a gap inherited from `distro`’s CLI being scaffolded thinner than `distroless`’s in Phase 0, not a limitation of `builder-core` itself.

04

THE STATIC BASE

Five packages exist purely to get from a mounted rootfs to a real, authenticated shell.

COREUTILS, BUSYBOX, AND UTILS – THREE ANSWERS TO THE SAME QUESTION

Every Unix system needs a basic set of file and text commands — `ls`, `cat`, `cp`, `mv`, `rm`, and so on. “Coreutils” is the generic name for that toolset, not one specific program. **GNU coreutils** is the implementation most desktop Linux distributions ship. **BusyBox** is a different, much older answer aimed at tiny/embedded systems: a single small C binary that crams coreutils **and** a shell, `init`, `mount`, `ps`, and dozens of other tools into one multi-call executable — this workspace’s other distro, `distroless`, uses it. **utils** is a third answer: a from-scratch reimplementa-tion of just coreutils, in Rust. This project uses utils instead of BusyBox for `distro` specifically because of the “prefer Rust alternatives to traditional tools” goal behind the whole `distro` crate — BusyBox would have worked technically, the same way it does for `distroless`.

The glibc equivalent of what BusyBox does as one binary is spread across five separate upstream projects here, because glibc-land has no single equivalent to reach for.

PACKAGE	VERSION	PROVIDES	WHY IT’S THERE
utils/coreutils	git main	<code>ls</code> , <code>cat</code> , <code>cp</code> , ...	A Rust reimplementa-tion of GNU coreutils — one multi-call binary, symlinked under every applet name. The built feature set is a curated subset (see §8), not the full set.
bash	5.2.37	<code>/bin/bash</code> , <code>/bin/sh</code>	The login shell named in <code>/etc/passwd</code> .
util-linux	2.41.2	<code>agetty</code> , <code>mount</code> , <code>umount</code>	Built with <code>--disable-all-programs</code> plus explicit <code>--enable-*</code> for just these three — util-linux ships dozens of tools, only these are needed.
shadow-utils	4.17.4	<code>login</code> , <code>passwd</code>	Real <code>/etc/passwd</code> + <code>/etc/shadow</code> authentication — not BusyBox’s separate empty-password mechanism, genuine shadow-file semantics.

All five are **statically linked** — `--disable-shared --enable-static` at configure time, `LDFLAGS=-all-static` at `make` time (plain `-static` alone breaks configure’s own compiler sanity check once `libtool` is involved). One binary compiled, one binary copied into the rootfs, nothing else to track. This is why they sit apart from everything in §5–6: dynamic linking doesn’t enter the picture until `dbus` (§5), which needs `libexpat` and isn’t meaningfully staticable.

A DISPATCH BUG SPECIFIC TO THIS BUILD

`utils` decides which applet `argv[0]` refers to by reading the kernel’s `AT_EXECFN` auxval on non-musl Linux (hardening against `argv[0]` spoofing) instead of trusting `argv[0]` directly. On this build host — and inside this project’s own kernel — `AT_EXECFN` comes back **empty**, so every applet invocation hit “<unknown binary name>” before dispatch even started. `distro/src/stages/fetch.rs` patches a fallback to `argv0` into `utils’ src/common/validation.rs` on every fetch, idempotently.

05 THE SEAT & SESSION LAYER

Three daemons exist to do the things a desktop session needs **before** any compositor can run: let an unprivileged process touch `/dev/input` and `/dev/dri`, pass messages between processes, and know what hardware is plugged in. None of them are optional for Phase 3 — they’re the floor a compositor stands on, not decoration.

WHAT UDEV ACTUALLY IS

The kernel’s device drivers know about hardware, but they don’t manage `/dev` themselves in any structured way — `devtmpfs` (already mounted by `distro-init`, §2) creates basic device nodes automatically as drivers load, and that’s all it does. `udev` is the userspace daemon that listens for the kernel’s own hardware-change announcements (sent over a netlink socket whenever something is plugged in, unplugged, or otherwise changes state), and in response creates or removes the matching `/dev` entries, applies permissions/rules, and maintains a live, queryable database of what hardware currently exists — the thing `udevadm info` reads from. Without it running, nothing gets notified when a device appears after boot, and nothing can ask “what’s connected right now” in a structured way. `eudev` is a fork of `udev` that works without `systemd`, which this project needs since it uses `seatd` instead of `systemd-logind` (below) — ordinary `udev` is a `systemd` subproject these days.

PACKAGE	VERSION	WHAT IT ACTUALLY DOES
<code>seatd</code>	0.9.3	Owns <code>/dev/input/*</code> and <code>/dev/dri/*</code> on behalf of whatever process asks nicely over its socket, so a compositor doesn’t need to run as root to touch a keyboard or a GPU. Its own README says it “depends only on libc” — the <code>systemd-logind</code> alternative, chosen explicitly over pulling in <code>systemd</code> (see the callout below).
<code>dbus</code>	1.16.2	The system message bus every desktop component uses to talk to every other one. Runs as root here (<code>-Ddbus_user=root</code>) — the rootfs has no unprivileged <code>messagebus</code> user yet to drop privileges to.
<code>eudev</code>	3.2.14	A <code>systemd</code> -independent fork of <code>udev</code> (what Alpine, Void, and Gentoo use without <code>systemd</code>) — walks <code>/sys</code> , builds a device database, and exposes it as <code>libudev</code> . Exists in this pipeline for one specific reason: <code>libinput</code> (§6) hard-depends on <code>libudev</code> , and there is no way around that dependency.

SEATD INSTEAD OF SYSTEMD — AN OPEN QUESTION, NOT A CLOSED ONE

Choosing `seatd` over `systemd` was a deliberate decision, made explicitly rather than defaulted into, to avoid pulling in `logind`, full `udev`, `journald`, and `networkd` for one seat-management socket. Whether `seatd` alone will actually satisfy COSMIC once Phase 5 needs a real session is still an open question — this system proves `seatd` starts and holds a seat, not that COSMIC will accept it as `logind`’s replacement.

`distro-init` starts all three (plus `udev`, which is really this same device-management job — see §2.1’s table) and supervises them the same way it supervises `agetty`. `dbus`’s socket lands at `/run/dbus/system_bus_socket` and `distro-init` creates `/run/dbus` itself immediately after mounting a fresh `tmpfs` at `/run`, since nothing else would.

06 THE WAYLAND-CORE LIBRARIES

Nothing in this section runs. Seven libraries are built and installed into the rootfs, waiting for Phase 3’s compositor to be the first thing that actually links against any of them — verification at this layer is “builds and installs cleanly against everything before it,” not “does something observable.”

PACKAGE	VERSION	WHAT IT’S FOR
<code>wayland</code>	1.26.0	The core wire-protocol libraries (client, server, cursor, EGL) and <code>wayland-scanner</code> , the code generator every later Wayland package runs at its own build time.

PACKAGE	VERSION	WHAT IT'S FOR
wayland-protocols	1.49	The actual protocol definitions — <code>xdg-shell</code> and the rest — as XML. No library of its own, just data plus a <code>pkg-config</code> file.
libxkbcommon	1.12.4	Turns “us, evdev, pc105” into the keymap tables a compositor hands to clients. X11 support is off (no <code>libxcb</code> — see §8); real keymap compilation needs the <code>xkeyboard-config</code> data package, also not built yet.
pixman	0.46.4	Software rasterization — Mesa’s fallback path, and some compositor-side operations not worth sending to the GPU.
libdisplay-info	0.4.0	Parses a monitor’s own EDID/DisplayID — how a compositor learns what resolutions and refresh rates a display actually supports.
libevdev	1.13.7	Reads and writes raw evdev input-device events. <code>libinput</code> ’s one mandatory dependency.
libinput	1.31.3	Turns raw evdev events into the pointer/keyboard/touch/gesture events a compositor actually wants. <code>libwacom</code> (tablets) and <code>mtdev</code> (legacy multitouch) are both left out — see §8.

The dependency order between them is also the build order: `wayland` first (nothing else here can build without `wayland-scanner`), then `wayland-protocols` (needs `wayland-scanner` on `PATH`), then the rest, with `libinput` last since it needs both `evdev`’s `libudev` (§5) and `libevdev`.

07 HOW IT’S ACTUALLY BUILT

7.1 THE CONFIG FILE

`distro.toml` is not one monolithic struct. `distro`’s `Config` composes the pieces that are genuinely identical to `distroless` (`KernelConfig`, `ImageConfig`, `UtilsConfig`, all from `builder-core`) with a `[section]` per package in §4–6 — each just a `version` and a source `url`, plus one `build_dir()` helper method per package computing exactly where its tarball extracts to.

7.2 THE PIPELINE, STAGE BY STAGE

DISTRO’S CLI SURFACE (`DISTRO/SRC/CLI.RS`)

```

distro fetch           # download + extract every source tarball
distro build-toolchain # apt-get the host build tools (once)
distro build-kernel    # builder-core, unchanged from distroless
distro build-userland  # every package in §4, §5, and §6
distro assemble-rootfs # merge it all into build-distro/rootfs
distro make-image      # partition + GRUB + write the disk image
distro test-qemu [--window] # boot it
distro write-usb --device <dev> # dd to real hardware (destructive)
distro all              # the whole pipeline, in order

```

`build-userland` is where §4–6’s packages actually compile — each package gets its own `build_<name>` function in `distro/src/stages/userland.rs`, called in dependency order. `assemble-rootfs` then builds the actual root filesystem tree: `coreutils` and its applet symlinks, `bash`, the §4 static binaries, the §5–6 dynamic ones (via the `sysroot`, below), the host’s own `libc.so.6` / `libexpat.so.1` / `libm.so.6` and dynamic linker (confirmed via `ld-linux-x86-64.so.2 --help` to already be on `glibc`’s default search path here — no `ldconfig` step needed), `distro-init` itself as `/sbin/init`, and `/etc/passwd` + `/etc/shadow`.

7.3 THE SYSROOT: HOW PACKAGES IN §5–6 FIND EACH OTHER

Phase 1’s five packages never needed each other at build time — each just needed the host’s `gcc`. §5–6’s packages do: `wayland-protocols` needs `wayland-scanner` on `PATH` at its own build time, and `libinput` needs `evdev`’s installed `libudev.pc` to link against `libudev` at all. Every package in §5–6

is therefore built with `--prefix=/usr` (its normal, final, “as if genuinely installed” prefix) and installed with `DESTDIR=<build-distro/sysroot>` — files physically land under the `sysroot`, but the package’s own compiled-in idea of its prefix stays `/usr`, which matters: `dbus`, for instance, looks up its own config file relative to whatever prefix it was **actually built with**, at its own runtime, on the real target — not at build time. `PKG_CONFIG_SYSROOT_DIR`, a mechanism `pkg-config` itself provides, then rewrites the `-I / -L` paths a later package’s build sees from `/usr/...` to the `sysroot`’s real, on-disk `<sysroot>/usr/...`. `assemble-rootfs` copies that whole `sysroot` tree into the final rootfs with one `cp -a`.

ONE PACKAGE BREAKS THIS PATTERN, ON PURPOSE

`wayland` alone is built differently: with a real, absolute, on-disk `--prefix=<sysroot>/usr` and installed directly, no `DESTDIR`. `wayland-scanner`’s own path is baked into `wayland`’s `.pc` file as a **custom** variable (`wayland_scanner=${bindir}/wayland-scanner`) that — unlike ordinary `Cflags/Libs` — `PKG_CONFIG_SYSROOT_DIR` does not rewrite, and `wayland-protocols`’ build executes whatever path that variable names, directly, as part of its own build. With `--prefix=/usr` that path would read the literal string `/usr/bin/wayland-scanner`, which does not exist anywhere on the host. This is safe specifically for `wayland` because none of its own **shared libraries** do a prefix-derived runtime lookup the way `dbus` / `eudev`’s daemons do.

An earlier version got this backwards — every package used the absolute `sysroot` prefix, not just `wayland` — and it booted, but `dbus-daemon` crash-looped: `distro-init` logged “`dbus-daemon` exited, respawning” on repeat, because `dbus` had baked in a literal build-machine path (`.../build-distro/sysroot/usr/share/dbus-1/system.conf`) as its config search location. Found by an actual QEMU boot, not by review.

7.4 HOW EVERY CLAIM IN THIS REPORT WAS CHECKED

A Python harness opens a PTY (`pty.openpty()`), with `TIOCSWINSZ` set — QEMU’s serial console renders nothing at a 0×0 terminal size), launches `qemu-system-x86_64` with the built image over `virtio` and `OVMF` for UEFI boot, and drives the console with a small state machine keyed on regexes over the accumulated output — `login:`, a shell prompt, a sentinel string echoed after each command — rather than fixed sleeps.

A BUILD-PIPELINE BUG THIS SAME METHOD CAUGHT

`assemble-rootfs` once failed outright with “No such file or directory” copying the freshly built `distro-init` binary. Cause: this build host sets `CARGO_TARGET_DIR` to a shared cache outside the repo, which `cargo` honors — so the binary was never at the hardcoded `target/...` path the `rootfs-assembly` code assumed. Fixed by reading `CARGO_TARGET_DIR` at the same call site `cargo` itself does.

08

WHAT ISN’T PART OF THE PICTURE YET

PHASE 3 Graphics, scoped first to QEMU’s `virtio-gpu`: `Mesa`, built against `glib`’s libraries.

PHASE 4 `rustup` / `cargo` on-target, plus a curated Rust-CLI-tools suite (`ripgrep`, `bat`, `eza`, ...).

PHASE 5 COSMIC itself — `cosmic-comp`, `cosmic-session`, `cosmic-panel`, `cosmic-greeter`, minimal subset first.

PHASE 6 Expand: more COSMIC components, real GPU drivers beyond `virtio-gpu`, audio, networking UI.

And three deliberate gaps in what’s already built, worth knowing about rather than discovering later:

- `utils`’ built feature set does not include `grep` or `sed`, despite both names appearing in `rootfs.rs`’s `applet-symlink` list — they were never part of `coreutils`’ scope upstream in the first place. The symlinks exist but dangle. Not yet fixed.
- `libxcb` was never built. Its only real consumer on a Wayland-native target is XWayland compatibility, not currently planned — three more from-source packages (`libxcb`, `libXau`, `libXdmcp`) not worth building for a checklist item with no near-term user.
- `mtdev` (legacy multitouch) and `libwacom` (tablet identification) were both left out of `libinput` — niche hardware support, easy to add back later if a real device needs it.